

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

Отчет по лабораторной работе №3

по дисциплине

«Объектно-ориентированное программирование»

Выполнил студент гр. ИВТб-2301-05-00

_____/Черкасов А. А./

Руководитель преподаватель

_____/Шмакова Н. А./

Киров 2026

Цель работы

Цель работы: изучить и освоить разработку серверных веб-приложений на языке Java с использованием фреймворка Spring Boot, механизмов объектно-реляционного отображения (ORM) на примере Hibernate для работы с базой данных PostgreSQL, а также проектирование REST API для взаимодействия с графическим веб-интерфейсом на базе React.

Задание

На основании согласованной в первой и второй лабораторных работах предметной области разработать REST API серверное приложение на Java с использованием Spring Boot. Сведения об объектах предметной области необходимо хранить в базе данных PostgreSQL. Серверное приложение должно выступать полноценной заменой C#-бэкенда и взаимодействовать с тем же графическим интерфейсом на React и базой данных.

Решение

Предметная область описывает управление гоночными командами трех различных серий (Formula 1, Formula 2, Formula E). Система позволяет хранить информацию о чемпионатах (сериях), гоночных командах и их пилотах, выполнять CRUD-операции, вычислять очки за позиции в гонке полиморфно по правилам конкретной серии, выполнять сравнение команд и предоставлять обобщенную статистику.

Программа содержит сущности JPA для отображения таблиц базы данных: «RacingSeries», «RacingTeam», «RacingDriver». Для передачи данных и поддержки полиморфизма на уровне API разработаны объекты передачи данных (DTO): «FormulaTeamDto» и «FormulaDriverDto».

Класс «RacingTeam» (сущность команды) содержит:

– Поля:

- private Long id – уникальный идентификатор (первичный ключ);
- private String name – название команды;

- private String principalName – имя руководителя;
- private String baseLocation – штаб-квартира;
- private String teamColor – фирменный цвет команды;
- private Integer points – набранные очки;
- private Integer wins – количество побед;
- private Integer podiums – количество подиумов;
- private RacingSeries series – связь со спортивной серией;
- private List<RacingDriver> drivers – список гонщиков команды;
- Специфичные поля серий: F1 (powerUnit, budgetCap, constructorPos), F2 (chassisModel, graduates, isFeeder), FE (energyPartner, batteryKwh, sustainScore);
- Методы: стандартные геттеры и сеттеры для всех полей.

Класс «RacingDriver» (сущность гонщика) содержит:

- Поля:
 - private Long id – уникальный идентификатор;
 - private String firstName – имя;
 - private String lastName – фамилия;
 - private Integer racingNumber – стартовый номер гонщика;
 - private Integer points – набранные личные очки;
 - private RacingTeam team – связь с командой;
- Методы: стандартные геттеры и сеттеры.

Класс «RacingSeries» (сущность серии) содержит:

- Поля:
 - private Long id – уникальный идентификатор;
 - private String name – название серии;
 - private String officialName – официальное название;
 - private String headquarters – штаб-квартира руководства;
 - private String governingBody – руководящий орган;
 - private Integer foundedYear – год основания;
 - private Integer rounds – количество этапов в календаре;

- private List<RacingTeam> teams – список команд серии;
- Методы: стандартные геттеры и сеттеры.

Диаграмма классов сущностей представлена на рисунке 1.

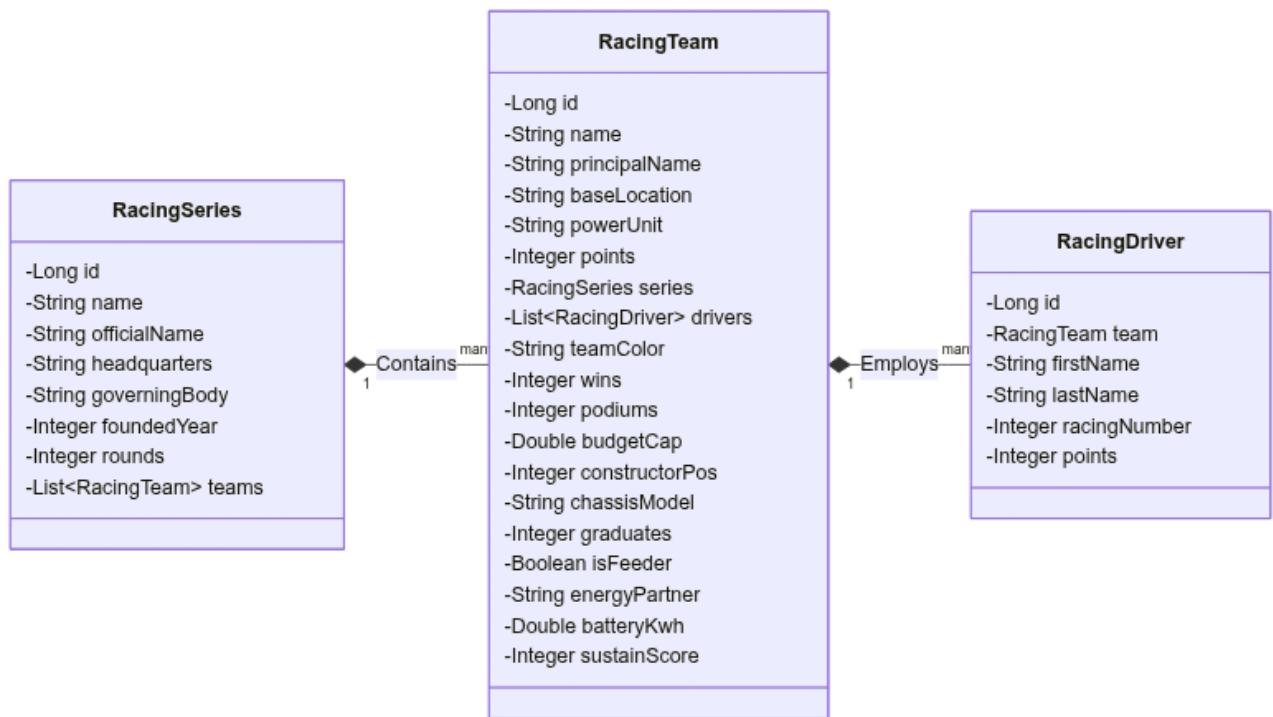


Рисунок 1 – Диаграмма классов сущностей

Для связывания Java-классов с таблицами реляционной СУБД PostgreSQL используются аннотации JPA/Hibernate:

- @Entity – объявляет класс сущностью базы данных;
- @Table(name = "...") – указывает имя соответствующей таблицы в СУБД;
- @Id и @GeneratedValue(strategy = GenerationType.IDENTITY) – определяют первичный ключ с автоинкрементом;
- @Column(name = "...") – задает имя колонки в таблице;
- @ManyToOne и @JoinColumn(name = "...") – настраивают связь «многие к одному» через внешний ключ (например, гонщик к команде);
- @OneToMany(mappedBy = "... cascade = CascadeType.ALL) – настраивает обратную связь «один ко многим» с каскадным удалением зависимых записей.

Подключение к базе данных PostgreSQL настраивается в файле application.properties:

```

spring.datasource.url=jdbc:postgresql://localhost:5432/postgres
spring.datasource.username=postgres
spring.datasource.password=postgres
  
```

```
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=validate
```

Для работы с сущностями в API используются репозитории Spring Data JPA, наследующие JpaRepository (например, RacingTeamRepository).

Для обработки HTTP-запросов и обмена данными с React-клиентом в JSON-формате разработан REST-контроллер «RacingRestController». Он реализует следующие эндпоинты:

- GET /api/teams – возвращает список Formula-команд (series_id 1..3), отсортированных по очкам;
- POST /api/teams – создает новую команду с сохранением в БД;
- PUT /api/teams/{id} – обновляет данные команды;
- DELETE /api/teams/{id} – удаляет команду и каскадно связанных гонщиков;
- GET /api/drivers – возвращает список Formula-гонщиков;
- POST /api/drivers – регистрирует нового гонщика;
- PUT /api/drivers/{id} – обновляет информацию о гонщике;
- DELETE /api/drivers/{id} – удаляет гонщика из системы;
- POST /api/teams/{id}/simulate-race – симулирует гонку, полиморфно добавляя очки команде и выбранному пилоту по правилам серии (15-12-10... для F2, 25-18-15... для F1/FE);
- GET /api/teams/compare – сравнивает две команды по очкам (имитируя перегруженные операторы C#);
- POST /api/simulation/race – глобальная симуляция гонки по переданному списку результатов;
- GET /api/stats – вычисляет общую статистику (лидеры, средние очки, количество участников) с фильтрацией по серии.

Диаграмма классов базы данных, репозитория и контроллера представлена на рисунке 2.

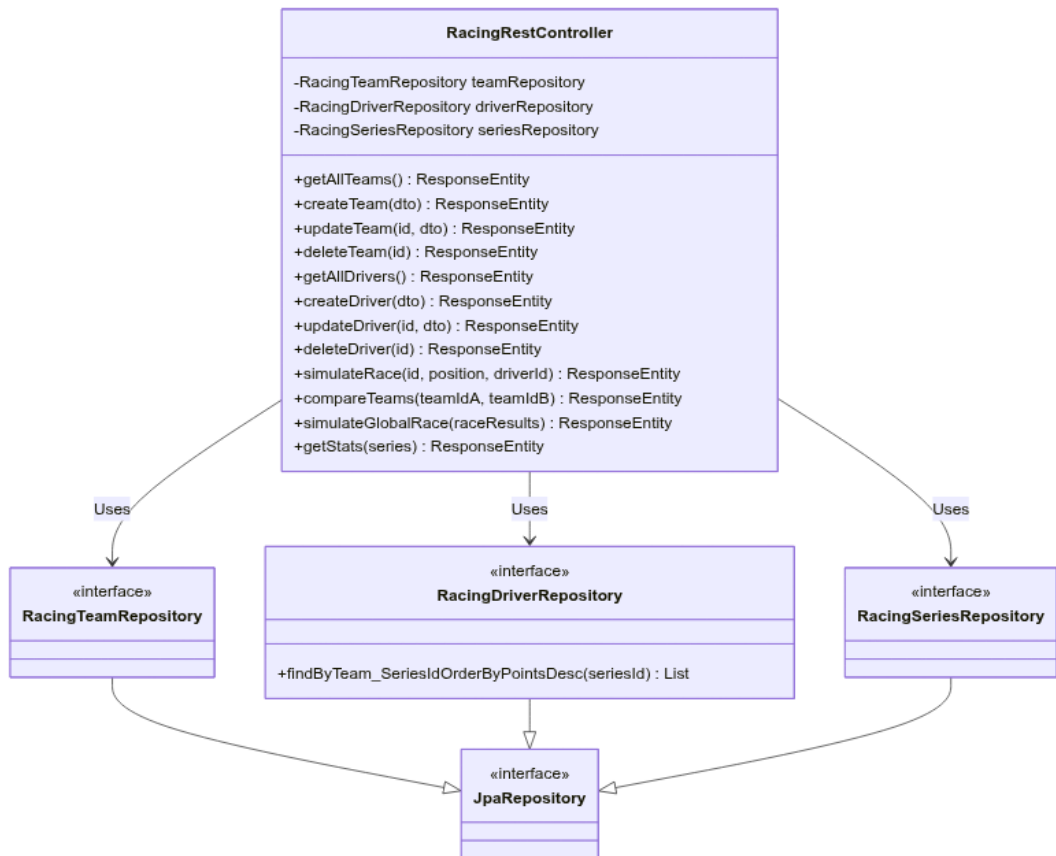


Рисунок 2 – Диаграмма классов репозиторий и REST-контроллера

Для авторизации запросов без необходимости прохождения аутентификации настроена конфигурация Spring Security в классе `SecurityConfig.java`.

Взаимодействие с веб-интерфейсом React полностью сохранено. Экранные формы представлены на рисунках 3, 4, 5, 6.

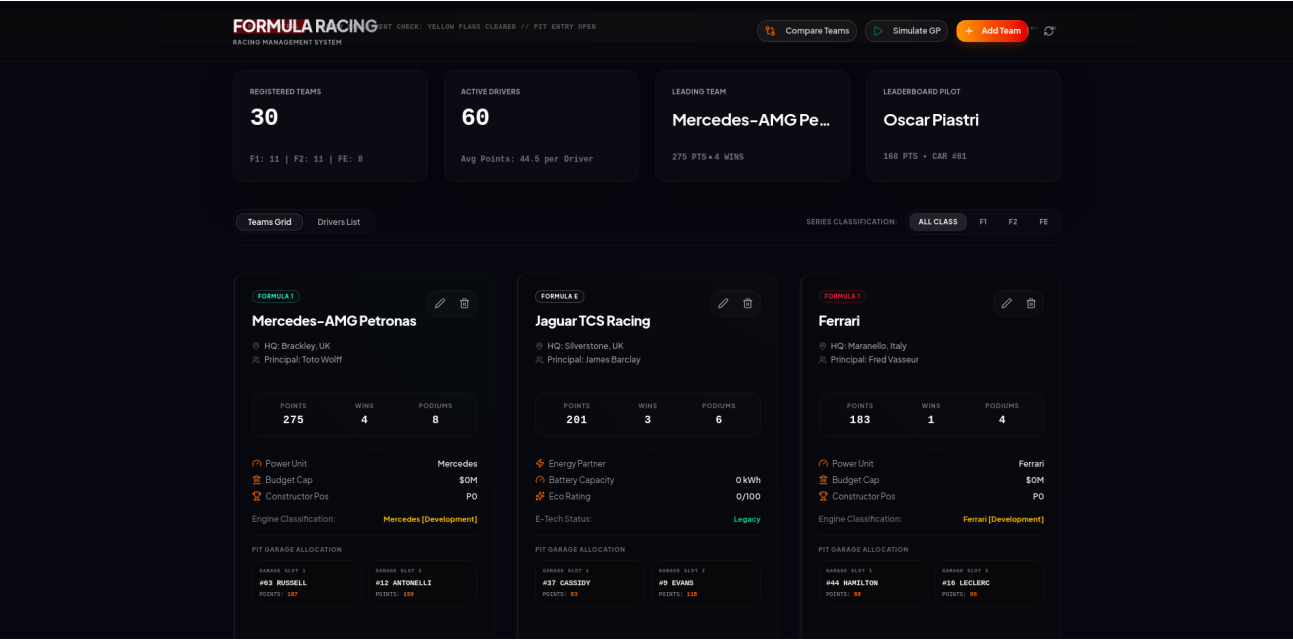


Рисунок 3 – Экранная форма вкладки команд (standings)

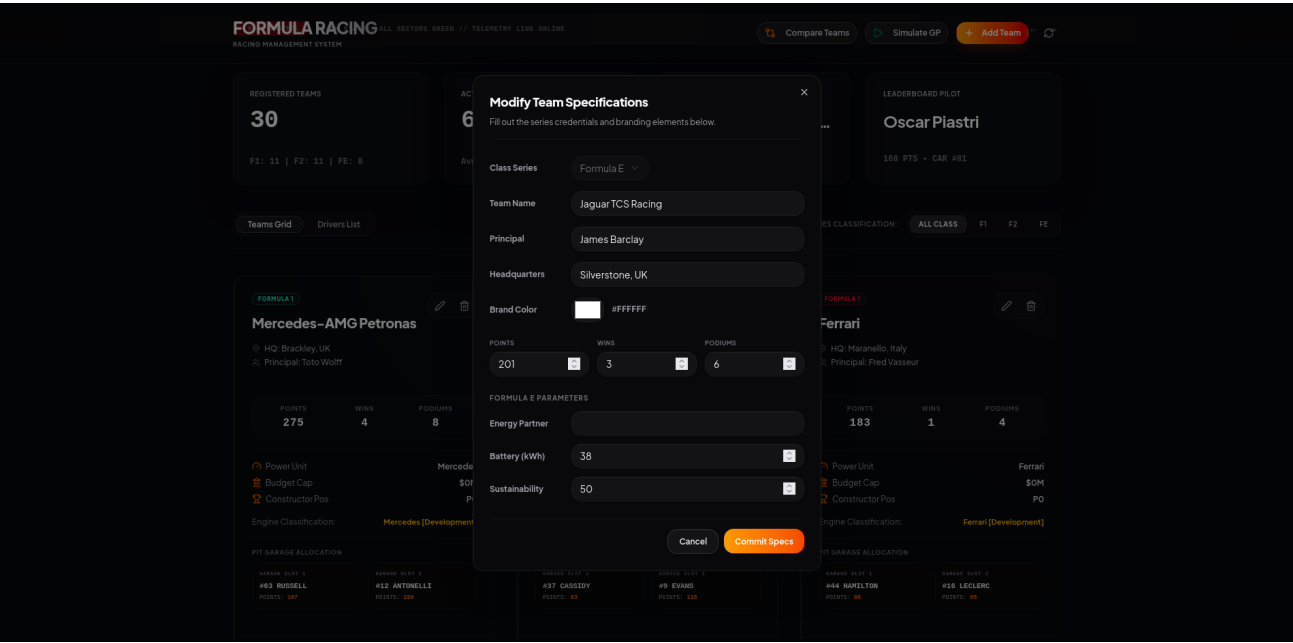


Рисунок 4 – Экранная форма вкладки гонщиков

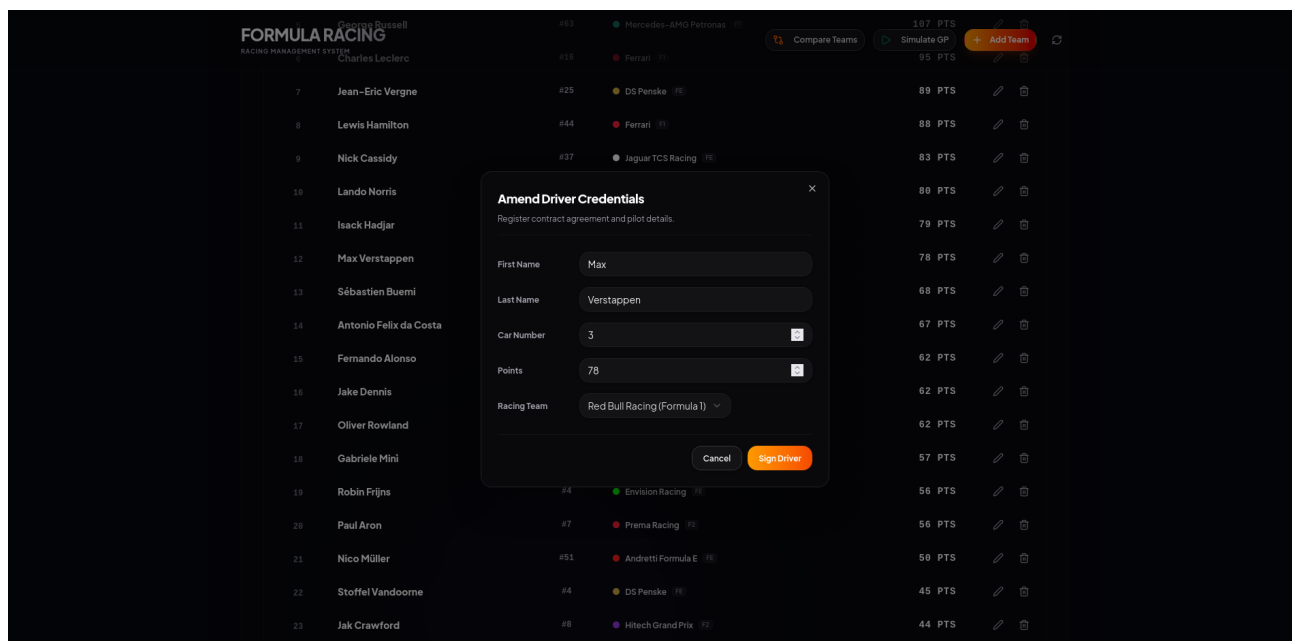


Рисунок 5 – Экранная форма создания и редактирования команд

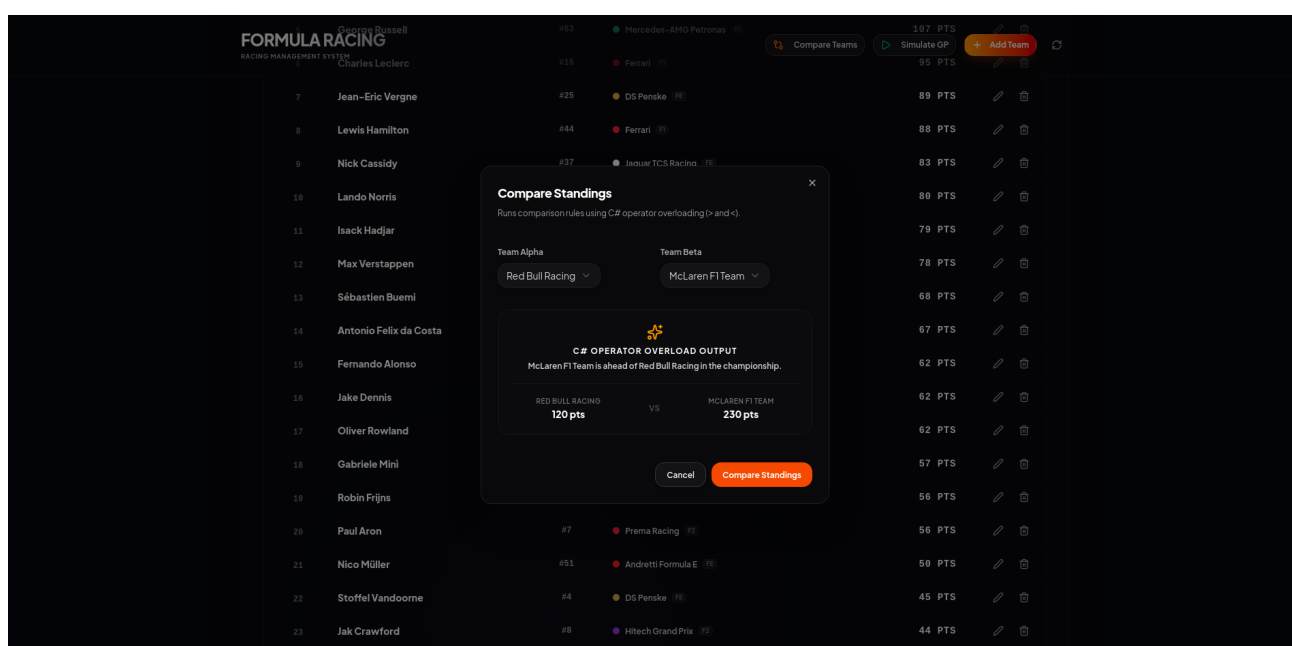


Рисунок 6 – Экранная форма создания и редактирования гонщиков

Вывод

В ходе выполнения лабораторной работы бэкенд гоночной системы управления был успешно перенесен с C# ASP.NET Core на Java Spring Boot с использованием Hibernate JPA и PostgreSQL. Была реализована идентичная REST API

маршрутизация и полиморфное поведение логики начисления очков, что позволило без каких-либо изменений на стороне клиента подключить разработанный React веб-интерфейс к новому Java-серверу. Были изучены принципы построения многослойной архитектуры в экосистеме Spring.

Приложение А1. Сущность RacingSeries.java

```
package com.vyatsu.racing.models;

import javax.persistence.*;
import java.util.List;

@Entity
@Table(name = "racing_series")
public class RacingSeries {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(name = "official_name")
    private String officialName;

    @Column(nullable = false)
    private String headquarters;

    @Column(name = "governing_body")
    private String governingBody;

    @Column(name = "founded_year")
    private Integer foundedYear;

    @Column(name = "rounds")
    private Integer rounds;

    @OneToMany(mappedBy = "series", cascade = CascadeType.ALL)
    private List<RacingTeam> teams;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
```

```

public void setName(String name) { this.name = name; }

public String getOfficialName() { return officialName; }
public void setOfficialName(String officialName) { this.officialName =
    ↪ officialName; }

public String getHeadquarters() { return headquarters; }
public void setHeadquarters(String headquarters) { this.headquarters =
    ↪ headquarters; }

public String getGoverningBody() { return governingBody; }
public void setGoverningBody(String governingBody) { this.governingBody =
    ↪ governingBody; }

public Integer getFoundedYear() { return foundedYear; }
public void setFoundedYear(Integer foundedYear) { this.foundedYear =
    ↪ foundedYear; }

public Integer getRounds() { return rounds; }
public void setRounds(Integer rounds) { this.rounds = rounds; }

public List<RacingTeam> getTeams() { return teams; }
public void setTeams(List<RacingTeam> teams) { this.teams = teams; }
}

```

Приложение А2. Сущность RacingTeam.java

```

package com.vyatsu.racing.models;

import lombok.Data;
import lombok.ToString;
import javax.persistence.*;
import java.util.List;

@Entity
@Table(name = "racing_teams")
public class RacingTeam {

    @Id

```

```

@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false)
private String name;

@Column(name = "principal_name", nullable = false)
private String principalName;

@Column(name = "base_location", nullable = false)
private String baseLocation;

@Column(name = "power_unit")
private String powerUnit;

@Column(nullable = false)
private Integer points = 0;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "series_id", nullable = false)
@ToString.Exclude
private RacingSeries series;

@OneToMany(mappedBy = "team", cascade = CascadeType.ALL)
@ToString.Exclude
private List<RacingDriver> drivers;

@Column(name = "team_color")
private String teamColor = "#FFFFFF";

// Lab3 V5: extra fields from lab2
@Column(nullable = false)
private Integer wins = 0;

@Column(nullable = false)
private Integer podiums = 0;

@Column(name = "budget_cap")
private Double budgetCap = 0.0;

```

```

@Column(name = "constructor_pos")
private Integer constructorPos = 0;

@Column(name = "chassis_model")
private String chassisModel;

@Column(nullable = false)
private Integer graduates = 0;

@Column(name = "is_feeder")
private Boolean isFeeder = false;

@Column(name = "energy_partner")
private String energyPartner;

@Column(name = "battery_kwh")
private Double batteryKwh = 0.0;

@Column(name = "sustain_score")
private Integer sustainScore = 0;

public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public String getPrincipalName() { return principalName; }
public void setPrincipalName(String principalName) { this.principalName =
    ↪ principalName; }

public String getBaseLocation() { return baseLocation; }
public void setBaseLocation(String baseLocation) { this.baseLocation =
    ↪ baseLocation; }

public String getPowerUnit() { return powerUnit; }
public void setPowerUnit(String powerUnit) { this.powerUnit = powerUnit; }

public Integer getPoints() { return points; }
public void setPoints(Integer points) { this.points = points; }

```

```

public String getTeamColor() { return teamColor; }
public void setTeamColor(String teamColor) { this.teamColor = teamColor; }

public RacingSeries getSeries() { return series; }
public void setSeries(RacingSeries series) { this.series = series; }

public List<RacingDriver> getDrivers() { return drivers; }
public void setDrivers(List<RacingDriver> drivers) { this.drivers = drivers; }

public Integer getWins() { return wins; }
public void setWins(Integer wins) { this.wins = wins; }

public Integer getPodiums() { return podiums; }
public void setPodiums(Integer podiums) { this.podiums = podiums; }

public Double getBudgetCap() { return budgetCap; }
public void setBudgetCap(Double budgetCap) { this.budgetCap = budgetCap; }

public Integer getConstructorPos() { return constructorPos; }
public void setConstructorPos(Integer constructorPos) { this.constructorPos =
    ↪ constructorPos; }

public String getChassisModel() { return chassisModel; }
public void setChassisModel(String chassisModel) { this.chassisModel =
    ↪ chassisModel; }

public Integer getGraduates() { return graduates; }
public void setGraduates(Integer graduates) { this.graduates = graduates; }

public Boolean getIsFeeder() { return isFeeder; }
public void setIsFeeder(Boolean isFeeder) { this.isFeeder = isFeeder; }

public String getEnergyPartner() { return energyPartner; }
public void setEnergyPartner(String energyPartner) { this.energyPartner =
    ↪ energyPartner; }

public Double getBatteryKwh() { return batteryKwh; }
public void setBatteryKwh(Double batteryKwh) { this.batteryKwh = batteryKwh; }

```

```

    public Integer getSustainScore() { return sustainScore; }
    public void setSustainScore(Integer sustainScore) { this.sustainScore =
        ↪ sustainScore; }
}

```

Приложение А3. Сущность RacingDriver.java

```

package com.vyatsu.racing.models;

import lombok.Data;
import lombok.ToString;
import javax.persistence.*;

@Entity
@Table(name = "racing_drivers")
public class RacingDriver {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", nullable = false)
    private String firstName;

    @Column(name = "last_name", nullable = false)
    private String lastName;

    @Column(name = "racing_number", nullable = false)
    private Integer racingNumber;

    @Column(nullable = false)
    private Integer points = 0;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "team_id", nullable = false)
    @ToString.Exclude
    private RacingTeam team;

    public Long getId() { return id; }
}

```

```

public void setId(Long id) { this.id = id; }

public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }

public String getLastName() { return lastName; }
public void setLastName(String lastName) { this.lastName = lastName; }

public Integer getRacingNumber() { return racingNumber; }
public void setRacingNumber(Integer racingNumber) { this.racingNumber =
    ↪ racingNumber; }

public Integer getPoints() { return points; }
public void setPoints(Integer points) { this.points = points; }

public RacingTeam getTeam() { return team; }
public void setTeam(RacingTeam team) { this.team = team; }
}

```

Приложение A4. Контроллер RacingRestController.java

```

package com.vyatsu.racing.controllers;

import com.vyatsu.racing.models.*;
import com.vyatsu.racing.repositories.*;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.*;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*", allowedHeaders = "*", methods = {RequestMethod.GET,
    ↪ RequestMethod.POST, RequestMethod.PUT, RequestMethod.DELETE,
    ↪ RequestMethod.OPTIONS})
public class RacingRestController {

```

```

private final RacingTeamRepository teamRepository;
private final RacingDriverRepository driverRepository;
private final RacingSeriesRepository seriesRepository;

public RacingRestController(RacingTeamRepository teamRepository,
                            RacingDriverRepository driverRepository,
                            RacingSeriesRepository seriesRepository) {
    this.teamRepository = teamRepository;
    this.driverRepository = driverRepository;
    this.seriesRepository = seriesRepository;
}

// --- TEAMS CRUD ---

@GetMapping("/teams")
public ResponseEntity<List<FormulaTeamDto>> getAllTeams() {
    List<RacingTeam> teams = new ArrayList<>();
    for (RacingTeam t : teamRepository.findAll()) {
        if (t.getSeries() != null) {
            long seriesId = t.getSeries().getId();
            if (seriesId >= 1 && seriesId <= 3) {
                teams.add(t);
            }
        }
    }

    // Sort descending by points (same as C# and React sorting)
    teams.sort((a, b) -> b.getPoints() - a.getPoints());

    List<FormulaTeamDto> dtos = teams.stream()
        .map(this::toTeamDto)
        .collect(Collectors.toList());
    return ResponseEntity.ok(dtos);
}

@PostMapping("/teams")
public ResponseEntity<FormulaTeamDto> createTeam(@RequestBody FormulaTeamDto
    ↪ dto) {
    RacingTeam team = new RacingTeam();
    updateTeamFromDto(team, dto);
}

```

```

        RacingTeam saved = teamRepository.save(team);
        return ResponseEntity.status(HttpStatus.CREATED).body(toTeamDto(saved));
    }

```

```

@PutMapping("/teams/{id}")
public ResponseEntity<FormulaTeamDto> updateTeam(@PathVariable Long id,
    ↪ @RequestBody FormulaTeamDto dto) {
    Optional<RacingTeam> optionalTeam = teamRepository.findById(id);
    if (!optionalTeam.isPresent()) {
        return ResponseEntity.notFound().build();
    }
    RacingTeam team = optionalTeam.get();
    updateTeamFromDto(team, dto);
    RacingTeam saved = teamRepository.save(team);
    return ResponseEntity.ok(toTeamDto(saved));
}

```

```

@DeleteMapping("/teams/{id}")
public ResponseEntity<Void> deleteTeam(@PathVariable Long id) {
    Optional<RacingTeam> optionalTeam = teamRepository.findById(id);
    if (!optionalTeam.isPresent()) {
        return ResponseEntity.notFound().build();
    }
    RacingTeam team = optionalTeam.get();
    // Cascade delete drivers signed under the team (managed by JPA cascade)
    teamRepository.delete(team);
    return ResponseEntity.noContent().build();
}

```

// --- DRIVERS CRUD ---

```

@GetMapping("/drivers")
public ResponseEntity<List<FormulaDriverDto>> getAllDrivers() {
    List<RacingDriver> drivers = new ArrayList<>();
    for (RacingDriver d : driverRepository.findAll()) {
        if (d.getTeam() != null && d.getTeam().getSeries() != null) {
            long seriesId = d.getTeam().getSeries().getId();
            if (seriesId >= 1 && seriesId <= 3) {
                drivers.add(d);
            }
        }
    }
}

```

```

    }
}

// Sort descending by points
drivers.sort((a, b) -> b.getPoints() - a.getPoints());

List<FormulaDriverDto> dtos = drivers.stream()
    .map(this::toDriverDto)
    .collect(Collectors.toList());
return ResponseEntity.ok(dtos);
}

@PostMapping("/drivers")
public ResponseEntity<FormulaDriverDto> createDriver(@RequestBody
↳ FormulaDriverDto dto) {
    RacingDriver driver = new RacingDriver();
    updateDriverFromDto(driver, dto);
    RacingDriver saved = driverRepository.save(driver);
    return
↳ ResponseEntity.status(HttpStatus.CREATED).body(toDriverDto(saved));
}

@PutMapping("/drivers/{id}")
public ResponseEntity<FormulaDriverDto> updateDriver(@PathVariable Long id,
↳ @RequestBody FormulaDriverDto dto) {
    Optional<RacingDriver> optionalDriver = driverRepository.findById(id);
    if (!optionalDriver.isPresent()) {
        return ResponseEntity.notFound().build();
    }
    RacingDriver driver = optionalDriver.get();
    updateDriverFromDto(driver, dto);
    RacingDriver saved = driverRepository.save(driver);
    return ResponseEntity.ok(toDriverDto(saved));
}

@DeleteMapping("/drivers/{id}")
public ResponseEntity<Void> deleteDriver(@PathVariable Long id) {
    Optional<RacingDriver> optionalDriver = driverRepository.findById(id);
    if (!optionalDriver.isPresent()) {
        return ResponseEntity.notFound().build();
    }
}

```

```

    }
    driverRepository.delete(optionalDriver.get());
    return ResponseEntity.noContent().build();
}

// --- SIMULATION & UTILITIES ---

@PostMapping("/teams/{id}/simulate-race")
public ResponseEntity<?> simulateRace(@PathVariable Long id,
                                     @RequestParam int position,
                                     @RequestParam Long driverId) {
    Optional<RacingTeam> optionalTeam = teamRepository.findById(id);
    if (!optionalTeam.isPresent()) {
        return
            ↳ ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of("error",
            ↳ "Team with ID " + id + " not found."));
    }
    RacingTeam team = optionalTeam.get();

    Optional<RacingDriver> optionalDriver =
        ↳ driverRepository.findById(driverId);
    if (!optionalDriver.isPresent() ||
        ↳ !optionalDriver.get().getTeam().getId().equals(id)) {
        return
            ↳ ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("error",
            ↳ "A valid driver signed with this team must be selected."));
    }
    RacingDriver driver = optionalDriver.get();

    int points = calculatePolymorphicPoints(team.getSeries().getId(),
        ↳ position);

    driver.setPoints(driver.getPoints() + points);
    team.setPoints(team.getPoints() + points);

    if (position == 1) {
        team.setWins(team.getWins() + 1);
        team.setPodiums(team.getPodiums() + 1);
    } else if (position <= 3) {

```

```

        team.setPodiums(team.getPodiums() + 1);
    }

    driverRepository.save(driver);
    RacingTeam savedTeam = teamRepository.save(team);

    FormulaTeamDto teamDto = toTeamDto(savedTeam);
    Map<String, Object> result = Map.of(
        "message", String.format("Simulated position P%d for %s %s (%d).",
            ↪ Calculated %d pts added to driver and team.",
            position, driver.getFirstName(), driver.getLastName(),
            ↪ driver.getRacingNumber(), points),
        "teamSummary", teamDto.summary,
        "team", teamDto
    );
    return ResponseEntity.ok(result);
}

@GetMapping("/teams/compare")
public ResponseEntity<?> compareTeams(@RequestParam Long teamIdA,
    ↪ @RequestParam Long teamIdB) {
    Optional<RacingTeam> optionalTeamA = teamRepository.findById(teamIdA);
    Optional<RacingTeam> optionalTeamB = teamRepository.findById(teamIdB);

    if (!optionalTeamA.isPresent() || !optionalTeamB.isPresent()) {
        return
            ↪ ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of("error",
            ↪ "One or both teams not found."));
    }

    RacingTeam teamA = optionalTeamA.get();
    RacingTeam teamB = optionalTeamB.get();

    int pointsA = teamA.getPoints();
    int pointsB = teamB.getPoints();

    boolean aIsGreater = pointsA > pointsB;
    boolean bIsGreater = pointsA < pointsB;

    String resultMessage = aIsGreater

```

```

        ? String.format("%s is ahead of %s in the championship.",
            ↪ teamA.getName(), teamB.getName())
        : bIsGreater
            ? String.format("%s is ahead of %s in the championship.",
                ↪ teamB.getName(), teamA.getName())
            : "Both teams have equal championship points.";

Map<String, Object> result = Map.of(
    "message", resultMessage,
    "teamA", Map.of("id", teamA.getId(), "name", teamA.getName(),
        ↪ "points", pointsA),
    "teamB", Map.of("id", teamB.getId(), "name", teamB.getName(),
        ↪ "points", pointsB),
    "winnerId", aIsGreater ? teamA.getId() : (bIsGreater ? teamB.getId() :
        ↪ 0L)
);
return ResponseEntity.ok(result);
}

```

```

@PostMapping("/simulation/race")
public ResponseEntity<?> simulateGlobalRace(@RequestBody
    ↪ List<DriverPositionResult> raceResults) {
    List<RacingTeam> teams = new ArrayList<>();
    teamRepository.findAll().forEach(teams::add);

    for (DriverPositionResult result : raceResults) {
        Optional<RacingDriver> optionalDriver =
            ↪ driverRepository.findById(result.getDriverId());
        if (!optionalDriver.isPresent()) continue;
        RacingDriver driver = optionalDriver.get();
        RacingTeam team = driver.getTeam();

        int points = calculatePolymorphicPoints(team.getSeries().getId(),
            ↪ result.getPosition());
        driver.setPoints(driver.getPoints() + points);
        team.setPoints(team.getPoints() + points);

        if (result.getPosition() == 1) {
            team.setWins(team.getWins() + 1);
        }
    }
}

```

```

        team.setPodiums(team.getPodiums() + 1);
    } else if (result.getPosition() <= 3) {
        team.setPodiums(team.getPodiums() + 1);
    }

    driverRepository.save(driver);
    teamRepository.save(team);
}

return ResponseEntity.ok(Map.of("message", "Global race simulation
    ↪ completed successfully. Driver and team standings updated."));
}

@GetMapping("/stats")
public ResponseEntity<?> getStats(@RequestParam(defaultValue = "all") String
    ↪ series) {
    List<RacingTeam> teams = new ArrayList<>();
    teamRepository.findAll().forEach(teams::add);
    List<RacingDriver> drivers = new ArrayList<>();
    driverRepository.findAll().forEach(drivers::add);

    List<RacingTeam> filteredTeams = teams.stream()
        .filter(t -> matchesSeries(t, series))
        .collect(Collectors.toList());

    List<RacingDriver> filteredDrivers = drivers.stream()
        .filter(d -> matchesSeries(d.getTeam(), series))
        .collect(Collectors.toList());

    RacingTeam leadingTeam = filteredTeams.stream()
        .max(Comparator.comparingInt(RacingTeam::getPoints))
        .orElse(null);

    RacingDriver leadingDriver = filteredDrivers.stream()
        .max(Comparator.comparingInt(RacingDriver::getPoints))
        .orElse(null);

    double avgPoints = filteredDrivers.stream()
        .mapToInt(RacingDriver::getPoints)
        .average()

```

```

        .orElse(0.0);

    // Round average points to 1 decimal place
    double roundedAvg = Math.round(avgPoints * 10.0) / 10.0;

    Map<String, Object> stats = Map.of(
        "registeredTeamsCount", filteredTeams.size(),
        "activeDriversCount", filteredDrivers.size(),
        "averagePoints", roundedAvg,
        "leadingTeamName", leadingTeam != null ? leadingTeam.getName() :
        ↪ "N/A",
        "leadingTeamPoints", leadingTeam != null ? leadingTeam.getPoints() :
        ↪ 0,
        "leadingTeamWins", leadingTeam != null ? leadingTeam.getWins() : 0,
        "leadingDriverName", leadingDriver != null ?
        ↪ (leadingDriver.getFirstName() + " " + leadingDriver.getLastName())
        ↪ : "N/A",
        "leadingDriverPoints", leadingDriver != null ?
        ↪ leadingDriver.getPoints() : 0,
        "leadingDriverNumber", leadingDriver != null ?
        ↪ leadingDriver.getRacingNumber() : 0
    );
    return ResponseEntity.ok(stats);
}

// --- MAPPING HELPERS ---

private boolean matchesSeries(RacingTeam t, String ser) {
    if (t == null || t.getSeries() == null) return false;
    long seriesId = t.getSeries().getId();
    if (seriesId < 1 || seriesId > 3) return false;
    if ("all".equalsIgnoreCase(ser)) return true;
    if ("f1".equalsIgnoreCase(ser) && seriesId == 1) return true;
    if ("f2".equalsIgnoreCase(ser) && seriesId == 2) return true;
    if ("fe".equalsIgnoreCase(ser) && seriesId == 3) return true;
    return false;
}

private int calculatePolymorphicPoints(long seriesId, int position) {

```

```

    if (position < 1 || position > 10) return 0;
    if (seriesId == 2) { // Formula 2
        int[] points = { 15, 12, 10, 8, 6, 5, 4, 3, 2, 1 };
        return points[position - 1];
    } else { // Formula 1 or Formula E
        int[] points = { 25, 18, 15, 12, 10, 8, 6, 4, 2, 1 };
        return points[position - 1];
    }
}

private FormulaTeamDto toTeamDto(RacingTeam team) {
    FormulaTeamDto dto = new FormulaTeamDto();
    dto.id = team.getId();
    dto.teamName = team.getName();
    dto.principalName = team.getPrincipalName();
    dto.headquarters = team.getBaseLocation();
    dto.team_color = team.getTeamColor();
    dto.championshipPoints = team.getPoints();
    dto.raceWins = team.getWins();
    dto.podiums = team.getPodiums();

    long seriesId = team.getSeries().getId();
    if (seriesId == 1) {
        dto.type = "f1";
        dto.seriesName = "Formula 1";
        dto.powerUnit = team.getPowerUnit();
        dto.budgetCapMln = team.getBudgetCap();
        dto.constructorPos = team.getConstructorPos();
        dto.engine_status = getPowerUnitStatus(team.getWins(),
            ↪ team.getPowerUnit());
        dto.summary = String.format("[Formula 1] %s | Pts: %d | Wins: %d | PU:
            ↪ %s | Budget: $%sM",
            dto.teamName, dto.championshipPoints, dto.raceWins,
            ↪ dto.powerUnit, dto.budgetCapMln);
    } else if (seriesId == 2) {
        dto.type = "f2";
        dto.seriesName = "Formula 2";
        dto.chassisModel = team.getChassisModel();
        dto.f1Graduates = team.getGraduates();
        dto.isFeederSeries = team.getIsFeeder();
    }
}

```

```

        dto.feeder_status_text = getFeederStatus(dto.isFeederSeries,
        ↪ dto.f1Graduates);
        dto.summary = String.format("[Formula 2] %s | Pts: %d | Wins: %d |
        ↪ Chassis: %s | F1 Grads: %d",
            dto.teamName, dto.championshipPoints, dto.raceWins,
            ↪ dto.chassisModel, dto.f1Graduates);
    } else if (seriesId == 3) {
        dto.type = "fe";
        dto.seriesName = "Formula E";
        dto.energyPartner = team.getEnergyPartner();
        dto.batteryCapacityKwh = team.getBatteryKwh();
        dto.sustainabilityScore = team.getSustainScore();
        dto.battery_rating_text =
        ↪ getBatteryEfficiencyRating(dto.batteryCapacityKwh);
        dto.summary = String.format("[Formula E] %s | Pts: %d | Wins: %d |
        ↪ Eco: %d/100 | Battery: %s kWh",
            dto.teamName, dto.championshipPoints, dto.raceWins,
            ↪ dto.sustainabilityScore, dto.batteryCapacityKwh);
    }

    dto.visible_color = getVisibleTeamColor(dto.team_color);

    if (team.getDrivers() != null) {
        dto.drivers = team.getDrivers().stream()
            .map(this::toDriverDto)
            .collect(Collectors.toList());
    } else {
        dto.drivers = new ArrayList<>();
    }

    return dto;
}

private FormulaDriverDto toDriverDto(RacingDriver driver) {
    FormulaDriverDto dto = new FormulaDriverDto();
    dto.id = driver.getId();
    dto.teamId = driver.getTeam().getId();
    dto.firstName = driver.getFirstName();
    dto.lastName = driver.getLastName();
}

```

```

        dto.racingNumber = driver.getRacingNumber();
        dto.points = driver.getPoints();
        dto.fullName = driver.getFirstName() + " " + driver.getLastName();
        dto.displayString = dto.fullName + " #" + driver.getRacingNumber();
        return dto;
    }

    private void updateTeamFromDto(RacingTeam team, FormulaTeamDto dto) {
        team.setName(dto.teamName);
        team.setPrincipalName(dto.principalName);
        team.setBaseLocation(dto.headquarters);
        team.setTeamColor(dto.team_color != null ? dto.team_color : "#ffffff");
        team.setPoints(dto.championshipPoints != null ? dto.championshipPoints :
            ↪ 0);
        team.setWins(dto.raceWins != null ? dto.raceWins : 0);
        team.setPodiums(dto.podiums != null ? dto.podiums : 0);

        String type = dto.type != null ? dto.type.toLowerCase() : "f1";
        long seriesId = "fe".equals(type) ? 3L : ("f2".equals(type) ? 2L : 1L);
        team.setSeries(seriesRepository.findById(seriesId).orElse(null));

        if ("f1".equals(type)) {
            team.setPowerUnit(dto.powerUnit != null ? dto.powerUnit : "Unknown");
            team.setBudgetCap(dto.budgetCapMln != null ? dto.budgetCapMln :
                ↪ 135.0);
            team.setConstructorPos(dto.constructorPos != null ?
                ↪ dto.constructorPos : 1);
        } else if ("f2".equals(type)) {
            team.setChassisModel(dto.chassisModel != null ? dto.chassisModel :
                ↪ "Dallara F2 2024");
            team.setGraduates(dto.f1Graduates != null ? dto.f1Graduates : 0);
            team.setIsFeeder(dto.isFeederSeries != null ? dto.isFeederSeries :
                ↪ true);
        } else if ("fe".equals(type)) {
            team.setEnergyPartner(dto.energyPartner != null ? dto.energyPartner :
                ↪ "Unknown");
            team.setBatteryKwh(dto.batteryCapacityKwh != null ?
                ↪ dto.batteryCapacityKwh : 38.0);
            team.setSustainScore(dto.sustainabilityScore != null ?
                ↪ dto.sustainabilityScore : 50);
        }
    }

```

```

    }
}

private void updateDriverFromDto(RacingDriver driver, FormulaDriverDto dto) {
    driver.setFirstName(dto.firstName);
    driver.setLastName(dto.lastName);
    driver.setRacingNumber(dto.racingNumber != null ? dto.racingNumber : 0);
    driver.setPoints(dto.points != null ? dto.points : 0);
    if (dto.teamId != null) {
        driver.setTeam(teamRepository.findById(dto.teamId).orElse(null));
    }
}

// --- DOMAIN STATUS STRINGS ---

private String getPowerUnitStatus(int wins, String powerUnit) {
    if (wins > 10) return powerUnit + " [Dominant]";
    if (wins > 4) return powerUnit + " [Competitive]";
    return powerUnit + " [Development]";
}

private String getFeederStatus(boolean isFeeder, int graduates) {
    if (!isFeeder) return "Independent";
    if (graduates >= 5) return "Elite Feeder (5+ F1 graduates)";
    if (graduates >= 2) return "Active Feeder";
    return "Developing Feeder";
}

private String getBatteryEfficiencyRating(double batteryKwh) {
    if (batteryKwh >= 40.0) return "Next-Gen (40+ kWh)";
    if (batteryKwh >= 38.0) return "Gen3 Standard (38 kWh)";
    return "Legacy";
}

private String getVisibleTeamColor(String hex) {
    if (hex == null || hex.trim().isEmpty()) return "#3b82f6";
    String cleanHex = hex.replace("#", "").trim();
    if (cleanHex.equalsIgnoreCase("000000") ||
        cleanHex.equalsIgnoreCase("000") ||
        cleanHex.equalsIgnoreCase("1e293b") ||

```

```

        cleanHex.equalsIgnoreCase("18181b")) {
            return "#a1a1aa";
        }
    if (cleanHex.length() == 3) {
        cleanHex = "" + cleanHex.charAt(0) + cleanHex.charAt(0) +
            cleanHex.charAt(1) + cleanHex.charAt(1) +
            cleanHex.charAt(2) + cleanHex.charAt(2);
    }
    if (cleanHex.length() != 6) return hex;
    try {
        int rVal = Integer.parseInt(cleanHex.substring(0, 2), 16);
        int gVal = Integer.parseInt(cleanHex.substring(2, 2), 16);
        int bVal = Integer.parseInt(cleanHex.substring(4, 2), 16);

        double r = rVal / 255.0;
        double g = gVal / 255.0;
        double b = bVal / 255.0;

        double luminance = 0.2126 * r + 0.7152 * g + 0.0722 * b;
        if (luminance < 0.35) {
            double max = Math.max(r, Math.max(g, b));
            double min = Math.min(r, Math.min(g, b));
            double h = 0, s = 0, l = (max + min) / 2.0;

            if (max != min) {
                double d = max - min;
                s = l > 0.5 ? d / (2.0 - max - min) : d / (max + min);
                if (max == r) {
                    h = (g - b) / d + (g < b ? 6.0 : 0.0);
                } else if (max == g) {
                    h = (b - r) / d + 2.0;
                } else if (max == b) {
                    h = (r - g) / d + 4.0;
                }
                h /= 6.0;
            }

            l = Math.max(l, 0.55);
            s = Math.max(s, 0.65);

```

```

        double q = l < 0.5 ? l * (1.0 + s) : l + s - l * s;
        double p = 2.0 * l - q;

        r = hue2Rgb(p, q, h + 1.0 / 3.0);
        g = hue2Rgb(p, q, h);
        b = hue2Rgb(p, q, h - 1.0 / 3.0);

        return String.format("#%02x%02x%02x",
            (int)Math.round(r * 255.0),
            (int)Math.round(g * 255.0),
            (int)Math.round(b * 255.0)).toLowerCase();
    }
} catch (Exception ex) {
    return hex;
}
return hex;
}

private double hue2Rgb(double p, double q, double t) {
    if (t < 0) t += 1.0;
    if (t > 1.0) t -= 1.0;
    if (t < 1.0 / 6.0) return p + (q - p) * 6.0 * t;
    if (t < 1.0 / 2.0) return q;
    if (t < 2.0 / 3.0) return p + (q - p) * (2.0 / 3.0 - t) * 6.0;
    return p;
}
}

```

Приложение А5. Объект FormulaTeamDto.java

```

package com.vyatsu.racing.models;

import java.util.List;

public class FormulaTeamDto {
    public Long id;
    public String type; // "f1", "f2", "fe"
    public String teamName;
    public String principalName;
}

```

```

public String headquarters;
public String team_color;
public Integer championshipPoints;
public Integer raceWins;
public Integer podiums;
public List<FormulaDriverDto> drivers;

// UI properties
public String seriesName;
public String summary;
public String visible_color;

// F1 properties
public String powerUnit;
public Double budgetCapMln;
public Integer constructorPos;
public String engine_status;

// F2 properties
public String chassisModel;
public Integer f1Graduates;
public Boolean isFeederSeries;
public String feeder_status_text;

// FE properties
public String energyPartner;
public Double batteryCapacityKwh;
public Integer sustainabilityScore;
public String battery_rating_text;
}

```

Приложение А6. Объект FormulaDriverDto.java

```

package com.vyatsu.racing.models;

public class FormulaDriverDto {
    public Long id;
    public Long teamId;
    public String firstName;

```

```

    public String lastName;
    public Integer racingNumber;
    public Integer points;
    public String fullName;
    public String displayString;
}

```

Приложение А7. Конфигурация SecurityConfig.java

```

package com.vyatsu.racing.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
    ↪ org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import
    ↪ org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter {

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests ()
                .antMatchers ("/**").permitAll () // Allow all endpoints for the
                ↪ lab
            .and ()
            .csrf ().disable ()
            .headers ().frameOptions ().disable ();
    }
}

```